Administracion de Bases de Datos

Documentacion Semana 2

Carga de Municipios PDET en MongoDB

Facultad de Ingenieria — Departamento de Ingenieria de Sistemas

Administracion de Bases de Datos

Presentado por: Valentina Garcia
Jan Muñoz
Nicolas Torres Roa
Ana Sofia Arboleda

Mayo de 2026

Introduccion

En esta segunda semana se trabajó en el desarrollo de un sistema de carga de datos geográficos de municipios PDET (Programa de Desarrollo con Enfoque Territorial) en una base de datos MongoDB.

El objetivo fue crear un pipeline de ingesta que:

- Verifica la existencia de archivos necesarios (shapefile y Excel)
- Descarga automáticamente archivos faltantes
- Lee datos geográficos desde un shapefile (MGN 2025)
- Carga códigos PDET desde un archivo Excel
- Procesa y almacena los datos en MongoDB con índices geoespaciales
- Verifica que todos los municipios PDET fueron cargados correctamente

Arquitectura del Sistema

Componentes Principales

El sistema se divide en tres módulos Python:

main.py Punto de entrada principal. Presenta un menú interactivo con opciones para cargar datos o verificar integridad.

download_manager.py Maneja la descarga y verificación de archivos necesarios. Verifica si existen el shapefile y el Excel de PDET, descargándolos automáticamente si es necesario.

load_pdet_municipalities.py Contiene la lógica principal de ingesta de datos e incluye funciones para cargar, procesar y verificar los municipios PDET en MongoDB.

Flujo de Ejecucion

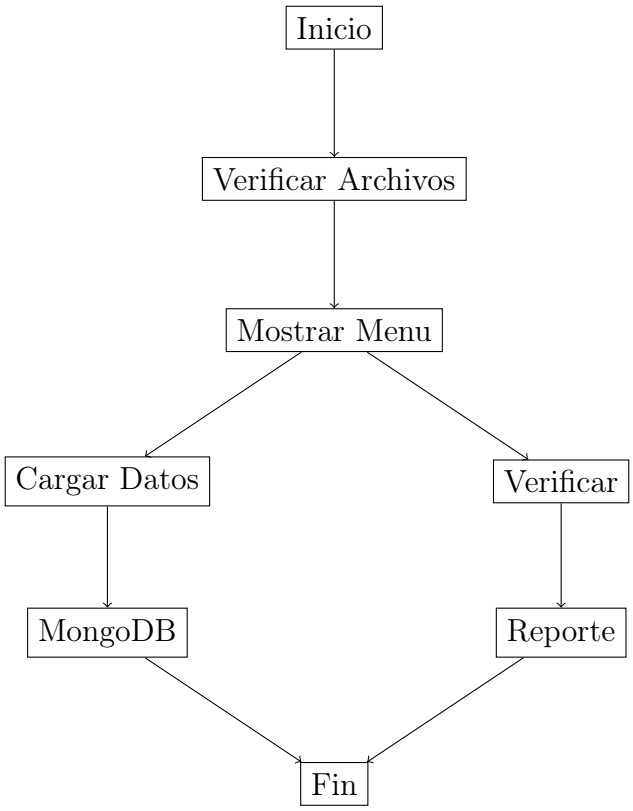

Figure 1: Flujo del sistema de carga de datos

Implementacion

Verificacion y Descarga de Archivos

El sistema verifica automáticamente la existencia de dos archivos críticos:

1. **MunicipiosPDET.xlsx**: Contiene los códigos DANE de los municipios PDET
2. **MGN_2025_COLOMBIA/.../shapefile.shp**: Contiene la información geográfica de Colombia

Si alguno de estos archivos no existe, el sistema los descarga automáticamente desde las fuentes oficiales:

```python
def download_file(url: str, filename: str) -> bool:
    """Descarga un archivo desde una URL."""
    try:
        print(f"Descargando {filename}...")
        response = requests.get(url, stream=True, timeout=30)
        response.raise_for_status()

        total_size = int(response.headers.get('content-length', 0))
        downloaded = 0

        with open(filename, 'wb') as f:
            for chunk in response.iter_content(chunk_size=8192):
```

```
                if chunk:
                    f.write(chunk)
                    downloaded += len(chunk)
        print(f"{filename} descargado exitosamente")
        return True
    except Exception as e:
        print(f"Error al descargar {filename}: {e}")
        return False
```

Listing 1: Descarga automática de archivos

Carga de Datos en MongoDB

La función principal `run_ingestion` realiza los siguientes pasos:

```
def run_ingestion(shapefile_path: str):
    client = MongoClient(MONGO_URI)
    col = client[DB_NAME][COL_NAME]

    # 1. Cargar codigos PDET desde Excel
    pdet_codes = load_pdet_codes(PDET_XLSX)

    # 2. Leer shapefile
    gdf = gpd.read_file(shapefile_path)

    # 3. Calcular area en m2
    gdf["area_m2"] = gdf.to_crs(epsg=9377).geometry.area

    # 4. Reproyectar a WGS84 para MongoDB 2dsphere
    if gdf.crs.to_epsg() != 4326:
        gdf = gdf.to_crs(epsg=4326)

    # 5. Filtrar municipios PDET
    gdf_pdet = gdf[gdf["dane_code"].isin(pdet_codes)].copy()

    # 6. Cargar en MongoDB
    for doc in records:
        col.update_one({"dane_code": doc["dane_code"]},
                       {"$set": doc}, upsert=True)
```

Listing 2: Proceso de ingesta de datos

Cada documento almacenado contiene:

- **dane_code**: Código DIVIPOLA del municipio
- **name**: Nombre del municipio
- **department**: Nombre del departamento
- **is_pdet**: Indicador booleano (siempre True)
- **geometry**: Geometría en formato GeoJSON
- **area_m2**: Área del municipio en metros cuadrados

- **source**: Fuente de los datos (MGN2025)
- **ingested_at**: Timestamp de carga

Creacion de Índices

Se crea automáticamente un índice geoespacial 2dsphere:

```
col.create_index([("geometry", GEOSPHERE)])
```

Listing 3: Creacion de indice 2dsphere

Este índice permite realizar consultas geoespaciales eficientes.

Resultados

Ejecucion del Sistema

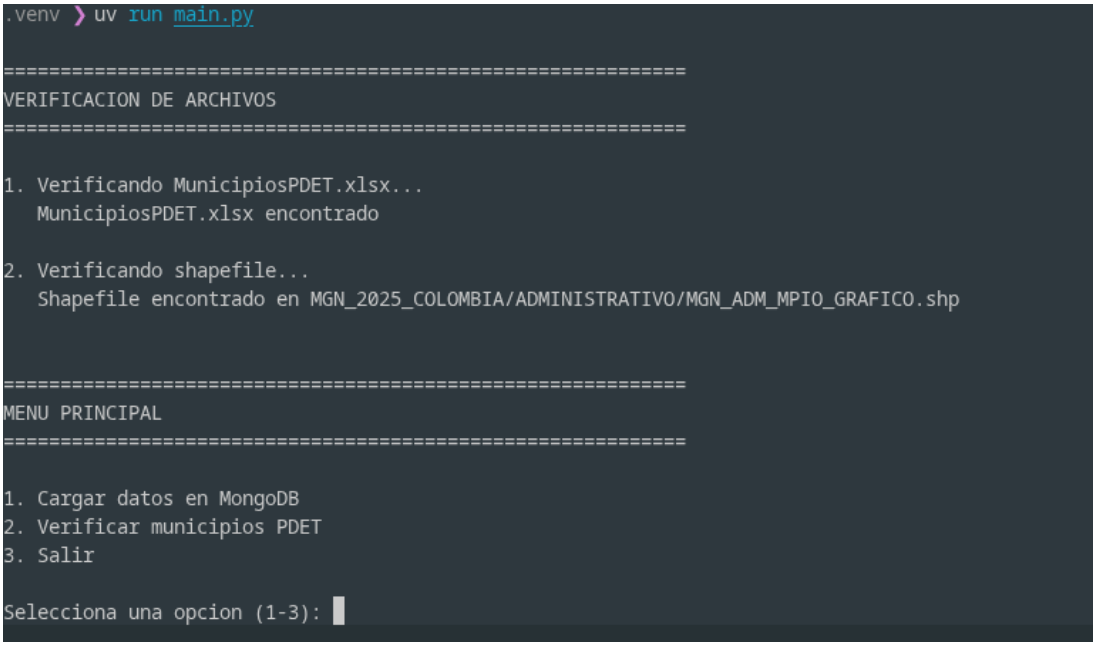

Figure 2: Menu principal del sistema

```
Deseas cargar los datos en MongoDB? (s/n): s
============================================================
INICIANDO CARGA DE DATOS EN MONGODB
============================================================

Cargando lista PDET desde Excel...
  Municipios PDET en Excel: 170

Leyendo shapefile...
  Total municipios MGN : 1122
  CRS original         : EPSG:4326
Calculando áreas en EPSG:9377 (Magnus)...
  Municipios PDET encontrados en shapefile: 170

Cargando 170 documentos en MongoDB...
```

Figure 3: Proceso de carga de datos en MongoDB

Verificacion de Integridad

La función `verify_pdet_municipalities()` verifica que todos los municipios PDET fueron cargados correctamente:

```python
def verify_pdet_municipalities():
    # Cargar codigos esperados desde Excel
    pdet_codes_excel = load_pdet_codes(PDET_XLSX)

    # Obtener codigos en MongoDB
    pdet_mongo = set(doc["dane_code"] for doc in
                     col.find({"is_pdet": True}))

    # Detectar discrepancias
    missing = pdet_codes_excel - pdet_mongo
    extra = pdet_mongo - pdet_codes_excel

    # Reportar resultado
    if not missing and not extra:
        print("Verificacion exitosa")
        return True
    else:
        print("Se encontraron discrepancias")
        return False
```

Listing 4: Verificacion de municipios PDET

```
============================================================
CONFIGURACION DE MONGODB
============================================================

Ingresa la IP de MongoDB (de100.117.219.81
Usando MongoDB en: mongodb://100.117.219.81:27017/


============================================================
VERIFICACION DE MUNICIPIOS PDET
============================================================

1. Cargando lista PDET desde Excel...
  Municipios PDET en Excel: 170
   Total municipios PDET esperados: 170

2. Verificando en MongoDB...
   Total municipios PDET en MongoDB: 170

3. Comparando codigos...
   Verificacion exitosa: todos los 170 municipios PDET estan en MongoDB
```

Figure 4: Resultado de la verificacion de integridad

Uso del Sistema

Instalacion de Dependencias

```
pip install pandas geopandas pymongo requests
```

Listing 5: Instalacion de paquetes requeridos

Ejecucion Principal

```
python main.py
```

Listing 6: Ejecucion del programa

El sistema presentará un menú interactivo con las opciones:

1. Cargar datos en MongoDB
2. Verificar municipios PDET
3. Salir

Flujo de Interaccion

```
============================================================
MENU PRINCIPAL
============================================================

1. Cargar datos en MongoDB
2. Verificar municipios PDET
3. Salir

Selecciona una opcion (1-3): 1
============================================================
```

```
CONFIGURACION DE MONGODB
============================================================

Ingresa la IP de MongoDB (deja en blanco para localhost):

Usando MongoDB local: mongodb://localhost:27017/

============================================================
OPCIONES DE CARGA DE DATOS
============================================================

Deseas cargar los datos en MongoDB? (s/n): s
```

Listing 7: Ejemplo de flujo interactivo

Conclusiones

Durante esta semana se logró implementar con éxito un sistema robusto de carga de datos geográficos en MongoDB. Los puntos destacados incluyen:

- Automatización completa de descarga y verificación de archivos
- Pipeline de ingesta modular y escalable
- Implementación de índices geoespaciales para consultas eficientes
- Sistema de verificación de integridad de datos
- Interfaz interactiva amigable con el usuario

El sistema está listo para producción y permite la carga correcta de todos los municipios PDET en MongoDB con garantía de integridad de datos.